

std::constant_wrapper

Document #: P2781R9
Date: 2025-06-17
Project: Programming Language C++
Audience: LEWG
LWG
Reply-to: Hana Dusíková
<hanicka@hanicka.net>
Matthias Kretz
<m.kretz@gsi.de>
Zach Laine
<whatwasthataddress@gmail.com>

Contents

- 1 Changelog
 - 1.1 Changes since R0
 - 1.2 Changes since R1
 - 1.3 Changes since R2
 - 1.4 Changes since R3
 - 1.5 Changes since R4
 - 1.6 Changes since R5
 - 1.7 Changes since R6
 - 1.8 Changes since R7
 - 1.9 Changes since R8
- 2 Relationship to previous work
- 3 The ergonomics of std::integral_constant<int> are bad
 - 3.1 Replacing the uses of std::integral_constant is not enough
 - 3.2 The difference in template parameters to std::constant_wrapper and std::cw
- 4 The type of X
- 5 Making constant_wrapper more useful
- 6 The problem with short-circuiting
- 7 The operator of questionable utility
- 8 What about strings?
- 9 An example using operator()
- 10 What about the mutating operators?
- 11 What about operator->?
- 12 Convertibility to and from std::integral_constant
- 13 Design

- 13.1 Add `constant_wrapper`
- 13.2 Tweak the definition of *integral-constant-like*
- 13.3 Add a feature macro
- 14 Implementation experience
- 15 Wording

§ 1 Changelog

§ 1.1 Changes since R0

- Remove discussion of literals for `std::integral_constant` and `std::constexpr_v`, based on LEWG feedback.
- Change the title to reflect the loss of the literals.
- Add an explicit example involving `std::constexpr_v::operator()`, as requested by LEWG.
- Add concept `std::constexpr_value`, as suggested by LEWG.
- Wording.

§ 1.2 Changes since R1

- Remove the `constexpr_value` concept.
- Use an auto NTTP parameter for `constexpr_v`, and default its type template parameter.
- Add the option for adding the mutating overloadable operators.
- Add a note about `operator->`.
- Add remarks about interconvertibility with `std::integral_constant`.
- Reduce the number of naming options.
- Simplify the implementation.

§ 1.3 Changes since R2

- Remove unnecessary uses of trailing return type.
- Remove use of `and`, `or` and `not` keywords.
- Correct the requirements in the exposition only *constexpr-param* concept.
- Add a note about the imperfect nature of ADL support given by `constexpr_v`'s `T` template parameter.

§ 1.4 Changes since R3

- Fix IFNDR in unary operator overloads (including `operator()` and `operator[]`).
- Add mutating operators, like `operator++` and `operator--`.
- Change the defaulted template parameter of the `constexpr_v` (formerly “`T`”) to be exposition-only.
- `constexpr_v -> constant_wrapper`, and `c_ -> cw`.

§ 1.5 Changes since R4

- Remove superfluous `inline` from `constexpr` variable templates. “Thanks,” Casey.
- Modified implementation to support array types, including string literals.

§ 1.6 Changes since R5

- Several wording corrections pointed out during LEWG review.

§ 1.7 Changes since R6

Changes suggested during LWG review in Hagenberg, and subsequent comments on the reflector. This includes:

- Remove “exposition-only” from the second param to `constant_wrapper`.
- Strike operator, .
- Express the semantics of *cw-fixed-value* in words instead of code.
- Add additional `!is_constructible_v<>` constraints to `operator&&` and `operator||`.
- Change the constraint on `operator()` from `invocable<value_type>` to `invocable<const value_type&>`.
- Change the definition of the exposition-only *integral-constant-like* in `[span.syn]` to be compatible with `constant_wrapper`.

§ 1.8 Changes since R7

Changes resulting from the telecon on 11 March, 2025.

- Re-introduce `operator,()`, but define it as deleted.
- Strike the `operator()` that returns `value` (not the one that returns a `constant_wrapper`).
- Strike the identity deduction guide for *cw-fixed-value*.

§ 1.9 Changes since R8

Changes requested in LWG review in Sofia on 17 June, 2025.

- type and data members of *cw-fixed-value* should be exposition-only.
- Change `constinit` to `constexpr` in multiple places.
- Remove the repeated declaration of `cw`.
- In the implementation of `operator->*`, remove “`e.operator`”.
- Strike paragraph 4 about `cw`’s purpose.
- Remove the “Constructors:” heading.
- Remove `constexpr` from variables in `impeccable_underground_planning()` in the example.
- Add a semicolon to the constructor definition.
- Italicize data in the “Effects:” clause.
- Add “freestanding” to the comment on the feature test macro.

§ 2 Relationship to previous work

This paper is co-authored in part by the authors of P2725R1 (“`std::integral_constant` Literals”) and P2772R0 (“`std::integral_constant` literals do not suffice — `constexpr_t`?”). This paper supersedes both of those previous papers.

§ 3 The ergonomics of `std::integral_constant<int>` are bad

`std::integral_constant<int>` is used in lots of places to communicate a constant integral value to a given interface. The length of its spelling makes it very verbose. Fortunately, we can do a lot better.

Before	After
<pre>// From P2630R1 auto const sir = std::strided_index_range{std::integral_constant<size_t, 0>{}, std::integral_constant<size_t, 10> {}, 3}; auto y = submdspan(x, sir);</pre>	<pre>auto y = submdspan(x, std::strided_index_range{ std::cw<0>, std::cw<10>, 3});</pre>

The “after” case above would require that `std::strided_index_range` be changed; that is not being proposed here. The point of the example is to show the relative convenience of `std::integral_constant` versus the proposed `std::constant_wrapper`.

§ 3.1 Replacing the uses of `std::integral_constant` is not enough

Parameters passed to a constexpr function lose their constexpr-ness when used inside the function. Replacing `std::integral_constant` with `std::constant_wrapper` has the potential to improve a lot more uses of compile-time constants than just integrals; what about all the other constexpr-friendly C++ types?

Consider:

```
template<typename T>
struct my_complex
{
    T re, im;
};

inline constexpr short foo = 2;

template<typename T>
struct S
{
    void f(auto c)
    {
        // c is to be used as a constexpr value here
    }
};
```

We would like to be able to call `S::f()` with a value, and have that value keep its constexpr-ness. Let’s introduce a template “constant_wrapper” that holds a constexpr value that it is given as an non-type template parameter.

```
namespace std {
    template</* ... */ X>
    struct constant_wrapper
    {
        using value_type = typename decltype(X)::type;
        using type = constant_wrapper;

        static constexpr const auto & value = X.data;

        constexpr operator decltype(auto)() const noexcept { return value; }

        // The rest of the members are discussed below ....
    };
}
```

Now we can write this.

```
template<typename T>
void g(S<T> s)
{
    s.f(std::constant_wrapper<1>{});
    s.f(std::constant_wrapper<2uz>{});
    s.f(std::constant_wrapper<3.0>{});
    s.f(std::constant_wrapper<4.f>{});
    s.f(std::constant_wrapper<foo>{});
    s.f(std::constant_wrapper<my_complex(1.f, 1.f)>{});
}
```

Let's now add a constexpr variable template with a shorter name, say cw.

```
namespace std {
    template</* ... */ X>
    constexpr constant_wrapper<X> cw{};
}
```

And now we can write this.

```
template<typename T>
void g(S<T> s)
{
    s.f(std::cw<1>);
    s.f(std::cw<2uz>);
    s.f(std::cw<3.0>);
    s.f(std::cw<4.f>);
    s.f(std::cw<foo>);
    s.f(std::cw<my_complex(1.f, 1.f)>);
}
```

§ 3.2 The difference in template parameters to std::constant_wrapper and std::cw

If you look at the wording below, you will see that std::cw takes a single NTTP, whereas std::constant_wrapper takes an NTTP X, and an exposition-only, unnamed parameter, which is defaulted to remove_cvref_t<decltype(X)>. Why is this? ADL, of course! Even though the type of X is deduced with or without the unnamed parameter, without the unnamed parameter, some natural uses of constant_wrapper cease to work. For instance:

```
auto f = std::cw<"foo">;
std::cout << f << "\n";
```

The stream insertion breaks without the unnamed parameter, which is char const [4], and which in turn pulls the proper operator<< into consideration during ADL. Note that this ADL support is imperfect. An earlier version of the paper showed using std::cw<strlit("foo")> in a stream insertion operation, where strlit is a structural type that contains the bytes that comprise "foo":

```
template<size_t N>
struct strlit
{
    constexpr strlit(char const (&str)[N]) { std::copy_n(str, N, value); }

    template<size_t M>
    constexpr bool operator==(strlit<M> rhs) const
    {
        return std::ranges::equal(bytes_, rhs.bytes_);
    }

    friend std::ostream & operator<<(std::ostream & os, strlit l)
    {
        assert(!l.value[N - 1] && "value must be null-terminated");
        return os.write(l.value, N - 1);
    }
}
```

```

    char value[N];
};

int main()
void print_foo()
{
    auto f = std::cw<strlit("foo")>;
    std::cout << f; // Prints "foo".
    std::cout << std::cw<"foo">; // Prints "foo".
}

```

This worked because of the way the operator<< above is declared – as a friend.

If it were instead declared as a non-friend:

```

template<size_t N>
std::ostream & operator<<(std::ostream & os, strlit<N> l) { /* ...*/ }

```

... ADL's help doesn't suffice. The deduction of N is not possible from a type that isn't a strlit<N> itself (e.g. base class) even if it is implicitly convertible to strlit<N>.

§ 4 The type of X

The type of X was elided above for simplicity; now let's look at it. This is what is used:

```

template<typename T>
struct fixed_value {
    using type = T;
    constexpr fixed_value(type v) noexcept: data(v) { }
    T data;
};

template<typename T, size_t Extent>
struct fixed_value<T[Extent]> {
    using type = T[Extent];
    constexpr fixed_value(T (&arr)[Extent]) noexcept: fixed_value(arr,
        std::make_index_sequence<Extent>()) { }
    T data[Extent];
private:
    template<size_t... Idx>
    constexpr fixed_value(T (&arr)[Extent], std::index_sequence<Idx...>) noexcept:
        data{arr[Idx]...} { }
};

template<typename T, size_t Extent>
fixed_value(T (&)[Extent]) -> fixed_value<T[Extent]>;
template<typename T>
fixed_value(T) -> fixed_value<T>;

```

By writing constant_wrapper as constant_wrapper<fixed_value X>, we are able to use CTAD in fixed_value X to defer writing the type of the underlying value, and use deduction to construct the write specialization of fixed_value at the point of specialization of constant_wrapper:

```

constexpr foo = constant_wrapper<"foo">;
static_assert(std::same_as<
    decltype(foo),
    const constant_wrapper<fixed_value<const char[4]>{"foo"}>>);
constexpr bar = constant_wrapper<42>;
static_assert(std::same_as<
    decltype(bar),
    const constant_wrapper<fixed_value<int>{42}>>);

```

This indirection allows the support for arrays that was added in this revision of the paper.

§ 5 Making constant_wrapper more useful

constant_wrapper is essentially a wrapper. It takes a value X of some structural type value_type, and represents X in such a way that we can continue to use X as a compile-time constant, regardless of context. As such, constant_wrapper should be implicitly convertible to value_type; this is already reflected in the design presented above. For the same reason, constant_wrapper should provide all the operations that the underlying type has. Though we cannot predict what named members the underlying type value_type has, we *can* guess at all the operator overloads it might have.

So, by adding conditionally-defined overloads for all the overloadable operators, we can make constant_wrapper as natural to use as many of the types it might wrap.

```
namespace std {
    struct operators {
        // unary -
        template<constexpr_param T>
            friend constexpr auto operator-(T) noexcept -> constant_wrapper<(-T::value)> { return
                {}; }

        // binary + and -
        template<constexpr_param L, constexpr_param R>
            friend constexpr auto operator+(L, R) noexcept -> constant_wrapper<(L::value +
                R::value)> { return {}; }
        template<constexpr_param L, constexpr_param R>
            friend constexpr auto operator-(L, R) noexcept -> constant_wrapper<(L::value -
                R::value)> { return {}; }

        // etc... (full listing later)
    };

    template<auto X>
    struct constant_wrapper : operators {
        using value_type = remove_cvref_t<decltype(X)>;
        using type = constant_wrapper;

        constexpr operator value_type() const { return X; }
        static constexpr value_type value = X;
    };
}
```

These operators are defined in such a way that they behave just like the operations on underlying the U and V values would, including promotions and coercions. For example:

```
static_assert(std::is_same_v<
    decltype(std::cw<42> - std::cw<13u>),
    std::constant_wrapper<29u>>);
```

Each operation is only defined if the underlying operation on X is defined. Each operation additionally requires that the result of the underlying operation have a structural type.

All the overloadable operations are included, even the index and call operators. The rationale for this is that a user may want to make some sort of compile-time domain-specific embedded language using operator overloading, and having all but a couple of the operators specified would frustrate that effort. The only exception to this is operator-> which must eventually return a pointer type, which is not very useful at compile time.

The operators are designed to interoperate with other types and templates that have a constexpr static value member. This works with std::constant_wrappers of course, but also std::integral_constants, and user-provided types as well. For example:

```
struct my_type { constexpr static int value = 42; };

void foo()
{
```

```
constexpr auto zero = my_type{} - std::cw<42>; // Ok.
// ...
}
```

Note that the addition of these operators is in line with the poll:

“Add a new robust integral constant type with all the numerical operators, as proposed in P2772R0, and use that for these literals instead of `std::integral_constant`”?

SF	F	N	A	SA
4	7	1	1	1

... taken in the 2023-01-17 Library Evolution telecon.

Note that the one SA said he would not be opposed if the word “integral” was stricken from the poll, and the design of `std::constant_wrapper` is not limited to integral types.

§ 6 The problem with short-circuiting

Consider this example, courtesy of Tim Song.

```
struct empty_range {
    constexpr int* begin() { return nullptr; }
    struct sentinel {
        friend constexpr auto operator==(int*, sentinel) { return std::cw<true>; }
        friend constexpr auto operator!=(int*, sentinel) { return std::cw<false>; }
    };
    constexpr auto end() {return sentinel{}}; }
};
auto pred = [] (int) { return std::cw<true>; };
template<class R, class P>
constexpr auto find_if(R&& r, P pred) {
    auto it = r.begin();
    while (it != r.end() && pred(*it)) {++it; }
    return it;
}
```

This fails to compile with the code presented in the previous version of this paper:

```
<source>: In function 'int main()':
<source>:211:12: error: call to consteval function 'find_if(empty_range, <lambda(int)> >
(empty_range(), (pred, <lambda(int)>()))' is not a constant expression
  211 |     find_if(empty_range{}, pred);
      |           ~~~~~~
<source>:211:12:   in 'constexpr' expansion of 'find_if(empty_range, <lambda(int)> >
(empty_range(), (pred, <lambda(int)>()))'
<source>:205:33: error: dereferencing a null pointer
  205 |     while (it != r.end() && pred(*it)) {++it; }
      |                               ~~~~~~
<source>:205:26: note: 'constexpr auto find_if(R&&, P) [with R = empty_range; P =
<lambda(int)>]' was promoted to an immediate function because its body contains an immediate-
escalating expression 'std::exposition_only::operator&&
<std::constant_wrapper<cw_fixed_value<bool>{0}, bool>,
std::constant_wrapper<cw_fixed_value<bool>{1}, bool> >((operator!=(it, ((& r)-
>empty_range::end(), empty_range::sentinel())),
std::constant_wrapper<std::exposition_only::cw_fixed_value<bool>{0}, bool>()), (pred.
<lambda(int)>)((* it)), std::constant_wrapper<std::exposition_only::cw_fixed_value<bool>{1},
bool>()))'
  205 |     while (it != r.end() && pred(*it)) {++it; }
      |           ~~~~~~
```

This failure occurs because `constant_wrapper` has overloads of `operator&&` and `operator||`, and user-defined overloads of these operators do not short circuit. However, for any type that is convertible to `bool`, we don't even

need the overloads. We only want to define the overload if it is needed by the underlying type (e.g. a type that is part of a DSEL) – and it is not when that value is a `bool`.

So, these operators now have an additional constraint. For example:

```
template<constexpr_param L, constexpr_param R>
    requires (!is_constructible_v<bool, decltype(L::value)> || !is_constructible_v<bool,
        decltype(R::value)>)
    friend constexpr auto operator&&(L, R) noexcept -> constant_wrapper<L::value &&
        R::value> { return {}; }
```

This makes the code in the `empty_range` example above well-formed.

§ 7 The operator of questionable utility

LWG reviewers noted that the `operator,` implementation is odd, in that it always results in an overloaded `constant_wrapper::operator,`. Here is its implementation.

```
template<constexpr_param L, constexpr_param R>
    friend constexpr auto operator,(L, R) noexcept -> constant_wrapper<L::value, R::value>
    { return {}; }
```

This is indeed silly. Since all the other operators are defined in terms of the well-formedness of the same operation on the underlying/wrapped values, it is odd to have this one operator always exist – which it will, since `expr1, expr2` is always well-formed for any two expressions `expr` and `expr2`.

One possible fix would be to change the definition to:

```
template<constexpr_param L, constexpr_param R>
    friend constexpr auto operator,(L, R) noexcept -> constant_wrapper<L::value.operator,
        (R::value)>
    { return {}; }
```

In this implementation, we only provide `operator,` if the underlying type `L::value` has it as a member function. The problem is that you could also define `operator,` as a free function. So the fix would break for free definitions of `operator,`, and would work for member definitions of `operator,`. Not great.

Since this operator is almost never used in practice, and since we can always add it later, we have removed it from the design.

§ 8 What about strings?

As mentioned above, in earlier versions of the paper, `std::cw<"foo">` did not work, because language rules prohibit using a reference to an array as an NTTP. However, the latest implementation uses an exposition-only structural type `cw-fixed-value` as the `constant_wrapper` NTTP; `cw-fixed-value` can be constructed from a variety of different types, including arrays. This allows an array to be given as the template parameter to `std::cw`, including an array of `char`, like a string literal. For instance:

```
void print_foo()
{
    std::cout << std::cw<"foo">; // Prints "foo".
}
```

This is not without its problems, however. Consider these two uses.

First, `std::cw<"foo"> == std::cw<"foo">`. This expression is true, but it only compares pointers.

Second, `std::cw<"bar"> < std::cw<"foo">` is not a constant expression. Runtime evaluation compares the pointers of “foo” and “bar”. That’s a footgun.

In general, strings and other array types are not passed nearly as often as arguments to constexpr functions. It is questionable whether we should delay the very useful semantics of `std::constexpr` function arguments trying to find a design that supports array types. This is a question that LEWG should answer.

Option 1: Support array values as NTTP template arguments to `std::constant_wrapper`. This is the option represented in the code that follows, based on LEWG polling.

Option 2: Leave all the code in place to support array value as NTTP template arguments to `std::constant_wrapper`, except disable the specialization that enables array values to work.

§ 9 An example using operator()

The addition of non-arithmetic operators may seem academic at first. However, consider this constexpr-friendly parser combinator mini-library.

```
namespace parse {

    template<typename L, typename R>
    struct or_parser;

    template<size_t N>
    struct str_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return lit == str_;
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<str_parser, P>{*this, parser};
        }
        strlit<N> str_;
    };

    template<typename L, typename R>
    struct or_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return l_(lit) || r_(lit);
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<or_parser, P>{*this, parser};
        }
        L l_;
        R r_;
    };

}

int foo()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p = p1 | p2 | p3;

    constexpr bool matches_empty = p(strlit(""));
}
```

```

static_assert(!matches_empty);
constexpr bool matches_pos = p(strlit("pos"));
static_assert(!matches_pos);
constexpr bool matches_decr = p(strlit("decr"));
static_assert(matches_decr);
}

```

(This relies on the `strlit` struct shown just previously.)

Say we wanted to use the templates in namespace `parser` along side other values, like `ints` and `floats`. We would want that not to break our `std::constant_wrapper` expressions. Having to work around the absence of `std::constant_wrapper::operator()` would require us to write a lot more code. Here is the equivalent of the function `foo()` above, but with all the variables wrapped using `std::cw`.

```

int bar()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p_ = std::cw<p1> | std::cw<p2> | std::cw<p3>;

    constexpr bool matches_empty_ = p_(std::cw<strlit("")>);
    static_assert(!matches_empty_);
    constexpr bool matches_pos_ = p_(std::cw<strlit("pos")>);
    static_assert(!matches_pos_);
    constexpr bool matches_decr_ = p_(std::cw<strlit("decr")>);
    static_assert(matches_decr_);
}

```

As you can see, everything works as it did before. The presence of `operator()` does not enable any new functionality, it just keeps code that happens to use it from breaking.

§ 10 What about the mutating operators?

It may seem at first that these operators are nonsensical, since all the operations on a `constant_wrapper` must be nonmutating.

However, some DSLs may wish to use these operations with atypical semantics.

```

struct weirdo
{
    constexpr int operator++() const { return 1; }
};
auto result = ++std::cw<weirdo{}>;

```

`result` is obviously `std::cw<1>` here, and no mutation occurred. You can imagine a more elaborate use case, say a library that is used to create expression templates. For example:

```

auto expr = std::cw<var0> += std::cw<var1>;

```

In this case, `var0` and `var1` would be some terminal types in the expression template library, and `operator+=` would return a `constexpr` expression tree, rather than mutating the left side of the `+=`.

These operators are now part of the proposal, based on this LEWG poll from Kona 2023:

“We should add mutating operations (i.e. `#define IF_LEWG_SAYS_S0 1` and `++` and `--`) to P2781R3”

SF	F	N	A	SA
2	6	5	2	0

§ 11 What about operator->?

We're not proposing it, because of its very specific semantics – it must yield a pointer, or something that eventually does. That's not a very useful operation during constant evaluation.

§ 12 Convertibility to and from `std::integral_constant`

During the LEWG reviews, some attendees suggested that inter-conversions between `std::integral_constant` and `std::constant_wrapper` would be useful. The important thing to remember is that we want deduction to occur when calling functions that take a `std::constant_wrapper`, including the `std::constant_wrapper` operator overloads. Conversions and deductions are at odds with one another, because deducing parameter types disables the conversion rules.

If you look at the operator overloads proposed here, you will see that they are deduction operations at their most essential. The types of the parameters do not matter, except that each conveys a value that is a core constant expression because it is embedded in the type system. The fact that a `std::constant_wrapper` conveys that value instead of a `std::integral_constant` is immaterial, and in fact the operators are written in such a way that they operate on either template (as long as at least one parameter is a specialization of `std::constant_wrapper`). Users can and should write their code using these kinds of values-as-types in a similar way. Relying on conversions is a less-useful way to get interoperability.

§ 13 Design

§ 13.1 Add `constant_wrapper`

```
namespace std {
    template<class T>
        struct cw-fixed-value; //
        exposition only

    template<cw-fixed-value X, class = typename decltype(cw-fixed-value(X))::type>
        struct constant_wrapper;

    template<class T>
        concept constexpr-param = requires { typename constant_wrapper<T::value>; }; //
        exposition only

    template<class T>
        struct cw-fixed-value { //
            exposition only
            using type = T; //
            exposition only
            constexpr cw-fixed-value(type v) noexcept: data(v) { } //
            T data; //
            exposition only
        };

    template<class T, size_t Extent>
        struct cw-fixed-value<T[Extent]> { //
            exposition only
            using type = T[Extent]; //
            exposition only
            constexpr cw-fixed-value(T (&arr)[Extent]) noexcept: cw-fixed-value(arr,
                std::make_index_sequence<Extent>()) { } //
            T data[Extent]; //
            exposition only

        private:
            template<size_t... Idx>
```

```

    constexpr cw-fixed-value(T (&arr)[Extent], std::index_sequence<Idx...>) noexcept:
    data{arr[Idx]...} { }
};

template<class T, size_t Extent>
    cw-fixed-value(T (&)[Extent]) -> cw-fixed-value<T[Extent]>;
    exposition only

struct cw-operators {
    exposition only
    // unary operators
    template<constexpr-param T>
        friend constexpr auto operator+(T) noexcept -> constant_wrapper<(+T::value)> { return
        {};}
    template<constexpr-param T>
        friend constexpr auto operator-(T) noexcept -> constant_wrapper<(-T::value)> { return
        {};}
    template<constexpr-param T>
        friend constexpr auto operator~(T) noexcept -> constant_wrapper<(~T::value)> { return
        {};}
    template<constexpr-param T>
        friend constexpr auto operator!(T) noexcept -> constant_wrapper<(!T::value)> { return
        {};}
    template<constexpr-param T>
        friend constexpr auto operator&(T) noexcept -> constant_wrapper<(&T::value)> { return
        {};}
    template<constexpr-param T>
        friend constexpr auto operator*(T) noexcept -> constant_wrapper<(*T::value)> { return
        {};}

    // binary operators
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator+(L, R) noexcept -> constant_wrapper<(L::value +
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator-(L, R) noexcept -> constant_wrapper<(L::value -
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator*(L, R) noexcept -> constant_wrapper<(L::value *
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator/(L, R) noexcept -> constant_wrapper<(L::value /
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator%(L, R) noexcept -> constant_wrapper<(L::value %
        R::value)> { return {};}

    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator<<(L, R) noexcept -> constant_wrapper<(L::value <<
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator>>(L, R) noexcept -> constant_wrapper<(L::value >>
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator&(L, R) noexcept -> constant_wrapper<(L::value &
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator|(L, R) noexcept -> constant_wrapper<(L::value |
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>
        friend constexpr auto operator^(L, R) noexcept -> constant_wrapper<(L::value ^
        R::value)> { return {};}

    template<constexpr-param L, constexpr-param R>
        requires (!is_constructible_v<bool, decltype(L::value)> || !is_constructible_v<bool,
        decltype(R::value)>))
        friend constexpr auto operator&&(L, R) noexcept -> constant_wrapper<(L::value &&
        R::value)> { return {};}
    template<constexpr-param L, constexpr-param R>

```

```

        requires (!is_constructible_v<bool, decltype(L::value)> || !is_constructible_v<bool,
decltype(R::value)>))
        friend constexpr auto operator||(L, R) noexcept -> constant_wrapper<(L::value ||
R::value)> { return {}; }

// comparisons
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator==(L, R) noexcept -> constant_wrapper<(L::value ==
R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<(L, R) noexcept -> constant_wrapper<(L::value <
R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<=(L, R) noexcept -> constant_wrapper<(L::value <=
R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator==(L, R) noexcept -> constant_wrapper<(L::value ==
R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator!=(L, R) noexcept -> constant_wrapper<(L::value !=
R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>(L, R) noexcept -> constant_wrapper<(L::value >
R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>=(L, R) noexcept -> constant_wrapper<(L::value >=
R::value)> { return {}; }

template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator,(L, R) noexcept = delete;
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator--*(L, R) noexcept -> constant_wrapper<L::value--*(
R::value)>
    { return {}; }

// call and index
template<constexpr-param T, constexpr-param... Args>
    constexpr auto operator()(this T, Args...) noexcept
        requires requires(Args...) { constant_wrapper<T::value(Args::value...)>(); }
    { return constant_wrapper<T::value(Args::value...)>{}; }
template<constexpr-param T, constexpr-param... Args>
    constexpr auto operator[](this T, Args...) noexcept ->
        constant_wrapper<(T::value[Args::value...])>
    { return {}; }

// pseudo-mutators
template<constexpr-param T>
    constexpr auto operator++(this T) noexcept requires requires(T::value_type x) { ++x; }
    { return constant_wrapper<[] { auto c = T::value; return ++c; }()>{}; }
template<constexpr-param T>
    constexpr auto operator++(this T, int) noexcept requires requires(T::value_type x) {
x++; }
    { return constant_wrapper<[] { auto c = T::value; return c++; }()>{}; }

template<constexpr-param T>
    constexpr auto operator--(this T) noexcept requires requires(T::value_type x) { --x; }
    { return constant_wrapper<[] { auto c = T::value; return --c; }()>{}; }
template<constexpr-param T>
    constexpr auto operator--(this T, int) noexcept requires requires(T::value_type x) { x-
-; }
    { return constant_wrapper<[] { auto c = T::value; return c--; }()>{}; }

template<constexpr-param T, constexpr-param R>
    constexpr auto operator+=(this T, R) noexcept requires requires(T::value_type x) { x +=
R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v += R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator-=(this T, R) noexcept requires requires(T::value_type x) { x -=
R::value; }

```

```

    { return constant_wrapper<[] { auto v = T::value; return v -= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator*=(this T, R) noexcept requires requires(T::value_type x) { x *=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v *= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator/=(this T, R) noexcept requires requires(T::value_type x) { x /=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v /= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator%=(this T, R) noexcept requires requires(T::value_type x) { x %=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v %= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator&=(this T, R) noexcept requires requires(T::value_type x) { x &=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v &= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator|=(this T, R) noexcept requires requires(T::value_type x) { x |=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v |= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator^=(this T, R) noexcept requires requires(T::value_type x) { x ^=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v ^= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator<=<=(this T, R) noexcept requires requires(T::value_type x) { x
    <=<= R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v <=<= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
constexpr auto operator>=>=(this T, R) noexcept requires requires(T::value_type x) { x
    >=>= R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v >=>= R::value; }()>{}; }
};

template<cw-fixed-value X, class>
struct constant_wrapper: cw-operators {
    static constexpr const auto & value = X.data;
    using type = constant_wrapper;
    using value_type = typename decltype(X)::type;

    template<constexpr-param R>
        constexpr auto operator=(R) const noexcept requires requires(value_type x) { x =
            R::value; }
        { return constant_wrapper<[] { auto v = value; return v = R::value; }()>{}; }

    constexpr operator decltype(auto)() const noexcept { return value; }
};

template<cw-fixed-value X>
constexpr auto cw = constant_wrapper<X>{};
}

```

§ 13.2 Tweak the definition of *integral-constant-like*

In [span.syn], there is an exposition-only concept, *integral-constant-like*. It needs to be adjusted slightly to change the `is_integral_v` part of the concept to `is_integral_v<remove_cvref_t<decltype(value)>>`. The required change is to make `constant_wrapper` compatible with *integral-constant-like*. This was suggested by one of the mdsbspan paper authors.

§ 13.3 Add a feature macro

Add a new feature macro, `__cpp_lib_constant_wrapper`.

§ 14 Implementation experience

One of the authors implemented the design in this paper. You can find it at:

https://github.com/tzlaine/constexpr/blob/master/include/constant_wrapper.hpp.

Additionally, an `integral_constant` with most of the operator overloads has been a part of [Boost.Hana](#) since its initial release in May of 2016. Its operations have been used by many, many users.

§ 15 Wording

Add the following to `[meta.type.synop]`, after `false_type`:

```
template<class T>
    struct cw-fixed-value;                                // exposition
    only

template<cw-fixed-value X, class = typename decltype(cw-fixed-value(X))::type>
    struct constant_wrapper;

template<class T>
    concept constexpr-param = requires { typename constant_wrapper<T::value>; }; // exposition
    only

struct cw-operators;                                     // exposition
    only

template<cw-fixed-value X>
    constexpr auto cw = constant_wrapper<X>{};
```

Add the following to `[meta.help]`, after `integral_constant`:

??? Class template `constant_wrapper` `[const.wrap.class]`

```
template<class T>
struct cw-fixed-value {                                    // exposition
    only
    using type = T;                                         // exposition
    only
    constexpr cw-fixed-value(type v) noexcept: data(v) { }
    T data;                                                 // exposition
    only
};

template<class T, size_t Extent>
struct cw-fixed-value<T[Extent]> {                          // exposition
    only
    using type = T[Extent];                                // exposition
    only
    constexpr cw-fixed-value(T (&arr)[Extent]) noexcept;
    T data[Extent];                                         // exposition
    only
};

template<class T, size_t Extent>
    cw-fixed-value(T (&)[Extent]) -> cw-fixed-value<T[Extent]>; // exposition
    only

struct cw-operators {                                       // exposition
    only
    // unary operators
    template<constexpr-param T>
        friend constexpr auto operator+(T) noexcept -> constant_wrapper<(+T::value)> { return {}; }
    }
    template<constexpr-param T>
        friend constexpr auto operator-(T) noexcept -> constant_wrapper<(-T::value)> { return {}; }
    }
    template<constexpr-param T>
```



```

    friend constexpr auto operator~(T) noexcept -> constant_wrapper<~T::value> { return {}; }
}
template<constexpr-param T>
    friend constexpr auto operator!(T) noexcept -> constant_wrapper<!T::value> { return {}; }
}
template<constexpr-param T>
    friend constexpr auto operator&(T) noexcept -> constant_wrapper<&T::value> { return {}; }
}
template<constexpr-param T>
    friend constexpr auto operator*(T) noexcept -> constant_wrapper<*T::value> { return {}; }
}

// binary operators
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator+(L, R) noexcept -> constant_wrapper<L::value + R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator-(L, R) noexcept -> constant_wrapper<L::value - R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator*(L, R) noexcept -> constant_wrapper<L::value * R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator/(L, R) noexcept -> constant_wrapper<L::value / R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator%(L, R) noexcept -> constant_wrapper<L::value % R::value>
    { return {}; }

template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<<(L, R) noexcept -> constant_wrapper<L::value <<
    R::value> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>>(L, R) noexcept -> constant_wrapper<L::value >>
    R::value> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator&(L, R) noexcept -> constant_wrapper<L::value & R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator|(L, R) noexcept -> constant_wrapper<L::value | R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator^(L, R) noexcept -> constant_wrapper<L::value ^ R::value>
    { return {}; }

template<constexpr-param L, constexpr-param R>
    requires (!is_constructible_v<bool, decltype(L::value)> || !is_constructible_v<bool,
    decltype(R::value)>)
    friend constexpr auto operator&&(L, R) noexcept -> constant_wrapper<L::value &&
    R::value> { return {}; }
template<constexpr-param L, constexpr-param R>
    requires (!is_constructible_v<bool, decltype(L::value)> || !is_constructible_v<bool,
    decltype(R::value)>)
    friend constexpr auto operator||(L, R) noexcept -> constant_wrapper<L::value ||
    R::value> { return {}; }

// comparisons
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<==(L, R) noexcept -> constant_wrapper<L::value <==
    R::value> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<(L, R) noexcept -> constant_wrapper<L::value < R::value>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<=(L, R) noexcept -> constant_wrapper<L::value <=
    R::value> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator==(L, R) noexcept -> constant_wrapper<L::value ==
    R::value> { return {}; }
template<constexpr-param L, constexpr-param R>

```

```

        friend constexpr auto operator!=(L, R) noexcept -> constant_wrapper<(L::value !=
        R::value)> { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>(L, R) noexcept -> constant_wrapper<(L::value > R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>=(L, R) noexcept -> constant_wrapper<(L::value >=
    R::value)> { return {}; }

template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator,(L, R) noexcept = delete;
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator->*(L, R) noexcept -> constant_wrapper<(L::value->*
    (R::value)>
    { return {}; }

// call and index
template<constexpr-param T, constexpr-param... Args>
    constexpr auto operator()(this T, Args...) noexcept
        requires requires(Args...) { constant_wrapper<T::value(Args::value...)>(); }
    { return constant_wrapper<T::value(Args::value...)>{}; }
template<constexpr-param T, constexpr-param... Args>
    constexpr auto operator[](this T, Args...) noexcept ->
        constant_wrapper<(T::value[Args::value...])>
    { return {}; }

// pseudo-mutators
template<constexpr-param T>
    constexpr auto operator++(this T) noexcept requires requires(T::value_type x) { ++x; }
    { return constant_wrapper<[] { auto c = T::value; return ++c; }()>{}; }
template<constexpr-param T>
    constexpr auto operator++(this T, int) noexcept requires requires(T::value_type x) { x++; }
    { return constant_wrapper<[] { auto c = T::value; return c++; }()>{}; }

template<constexpr-param T>
    constexpr auto operator--(this T) noexcept requires requires(T::value_type x) { --x; }
    { return constant_wrapper<[] { auto c = T::value; return --c; }()>{}; }
template<constexpr-param T>
    constexpr auto operator--(this T, int) noexcept requires requires(T::value_type x) { x--; }
    { return constant_wrapper<[] { auto c = T::value; return c--; }()>{}; }

template<constexpr-param T, constexpr-param R>
    constexpr auto operator+=(this T, R) noexcept requires requires(T::value_type x) { x +=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v += R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator-=(this T, R) noexcept requires requires(T::value_type x) { x -=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v -= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator*=(this T, R) noexcept requires requires(T::value_type x) { x *=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v *= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator/=(this T, R) noexcept requires requires(T::value_type x) { x /=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v /= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator%=(this T, R) noexcept requires requires(T::value_type x) { x %=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v %= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator&=(this T, R) noexcept requires requires(T::value_type x) { x &=
    R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v &= R::value; }()>{}; }
template<constexpr-param T, constexpr-param R>

```

```

    constexpr auto operator|=(this T, R) noexcept requires requires(T::value_type x) { x |=
        R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v |= R::value; }>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator^=(this T, R) noexcept requires requires(T::value_type x) { x ^=
        R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v ^= R::value; }>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator<=<=(this T, R) noexcept requires requires(T::value_type x) { x <=
        R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v <= R::value; }>{}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator>=>=(this T, R) noexcept requires requires(T::value_type x) { x >=
        R::value; }
    { return constant_wrapper<[] { auto v = T::value; return v >= R::value; }>{}; }
};

template<cw-fixed-value X, class>
struct constant_wrapper: cw-operators {
    static constexpr const auto & value = X.data;
    using type = constant_wrapper;
    using value_type = typename decltype(X)::type;

    template<constexpr-param R>
        constexpr auto operator=(R) const noexcept requires requires(value_type x) { x = R::value;
        }
        { return constant_wrapper<[] { auto v = value; return v = R::value; }>{}; }

    constexpr operator decltype(auto)() const noexcept { return value; }
};

```

- 2 The class template `constant_wrapper` aids in metaprogramming by ensuring that the evaluation of expressions comprised entirely of `constant_wrappers` are core constant expressions (`[expr.const]`), regardless of the context in which they appear. In particular, this enables use of `constant_wrapper` values that are passed as arguments to `constexpr` functions to be used in constant expressions.

- 3 [*Note*: The unnamed second template parameter to `constant_wrapper` is present to aid argument-dependent lookup (6.5.4 [\[basic.lookup.argdep\]](#)) in finding overloads for which `constant_wrapper`'s wrapped value is a suitable argument, but for which the `constant_wrapper` itself is not. — *end note*]

- 4 [*Example*:

```

constexpr auto initial_phase(auto quantity_1, auto quantity_2) {
    return quantity_1 + quantity_2;
}

constexpr auto middle_phase(auto tbd) {
    return tbd;
}

void final_phase(auto gathered, auto available) {
    if constexpr (gathered == available)
        std::cout << "Profit!\n";
}

void impeccable_underground_planning() {
    auto gathered_quantity = middle_phase(initial_phase(std::cw<42>, std::cw<13>));
    static_assert(gathered_quantity == 55);
    auto all_available = std::cw<55>;
    final_phase(gathered_quantity, all_available);
}

void deeply_flawed_underground_planning() {
    constexpr auto gathered_quantity = middle_phase(initial_phase(42, 13));
    constexpr auto all_available = 55;
}

```

```

        final_phase(gathered_quantity, all_available); // error: 'gathered == available' is not a
            constant expression
    }

```

— end example]

```
constexpr cw-fixed-value(T (&arr)[Extent]) noexcept;
```

¹ *Effects:* Initialize elements of *data* with corresponding elements of *arr*.

Change [span.syn]:

```

template<class T>
    concept integral-constant-like =                // exposition only
-   is_integral_v<decltype(T::value)>
+   is_integral_v<remove_cvref_t<decltype(T::value)>> &&
    !is_same_v<bool, remove_const_t<decltype(T::value)>> &&
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value::value> &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value::value>;

```

Add to [version.syn]:

```
#define __cpp_lib_constant_wrapper XXXXXL // freestanding, also in <type_traits>
```